

2 Section B

In this section, you implement an **end-to-end retrieval pipeline** over a collection of textual Wikipedia-style entries. Your system shall receive a **batch of queries** and return, for each query, a ranked list of relevant `page_id` values.

Unlike Section A (dynamic vector *index* operations), you own the full pipeline: preprocessing, optional chunking, embedding, offline indexing, and query-time retrieval. The corpus is partially synthetic; facts may be fictional. Queries are built over this corpus so the complete relevant set per query is known. A query may have **multiple** relevant pages.

Student package – files you receive Download the Section B bundle from link below. Unpack the `student/` folder. You should have:

- `main.py` – implements `run(queries)` called by the autograder.
- `chunk.py`, `embed.py`, `index.py`, `retrieve.py`, `utils.py` – pipeline modules you may edit.
- `eval.py` – **read-only** NDCG@10 utilities.
- `requirements.txt`, `README.md`.
- `scripts/eval_public.py` – self-test on 50 public queries (**read-only**).
- `scripts/build_index.py` – local offline index build (**read-only**; for your machine only).
- `data/public_queries.json` – 50 public queries with labels.
- `data/Wikipedia Entries/` – one JSON file per page (full retrieval corpus).
- `artifacts/` – directory for your precomputed index and embeddings (empty in the handout; **required in your submission**).

Not included in the student zip:

- `hidden_queries.json` – instructor/autograder only (you never receive it).

You may modify every file in `student/` **except** `eval.py`, `scripts/eval_public.py`, and `scripts/build_index.py`.

Section B Download Link

Corpus and JSON format Each page file:

```
{
  "page_id": 25051,
  "title": "...",
  "content": "..."
}
```

`page_id` is an integer. Public queries:

```
{
  "query_id": "q_public_001",
  "query": "...",
  "relevant_page_ids": [20263, 9112]
}
```

Relevance is binary. Retrieval is over **all** files in `data/Wikipedia Entries/`.

Required API The autograder calls **once** with all evaluation queries:

```
def run(queries: list[str]) -> list[list[int]]:
```

Return one ranked list of **page_id** (**int**) per query, most relevant first. Only the first 10 IDs per list are scored. Duplicate IDs after the first occurrence and invalid IDs earn no extra gain.

Embeddings and imports Use `sentence-transformers/all-MiniLM-L6-v2` for all retrieval embeddings.

Import restriction: only the standard library and `requirements.txt`: `numpy`, `sentence-transformers`, `faiss-cpu` (or `faiss`).

Submitted index (required) You must build the corpus index **offline on your own machine** and submit the resulting files under `artifacts/` in your public GitHub repository. The autograder **does not** run `scripts/build_index.py` or call `build_offline_index()` at grading time; it only calls `run(queries)` and expects your code to load the prebuilt index from disk.

Include every file your `run()` implementation needs (e.g. vector matrices, FAISS indexes, metadata JSON). Document each artifact path and format in your README. Custom **trained** checkpoints must be in the repo; pretrained MiniLM weights need not be shipped if loaded from the hub. Use Git LFS if files are large.

Local setup: document `pip install -r requirements.txt` and index build in your README for your own machine. **Grading** assumes dependencies are installed and only runs your code (see GitHub rubric below).

Pipeline and timing Design freely (chunking, FAISS, reranking, etc.). Grading is at `page_id` level.

- **Offline (untimed, your machine):** corpus embedding and index build (`python scripts/build_index.py`). Not run by staff at grading time.
- **Graded (≤ 60 s, GPU):** one call `run(all_queries)` including query embedding and retrieval over your submitted `artifacts/`.

Self-evaluation

```
pip install -r requirements.txt
python scripts/build_index.py
python scripts/eval_public.py
```

Prints mean NDCG@10 on public queries only. Hidden queries are not distributed.

Section B grade Your Section B score has three weighted components; creativity is judged separately as a possible bonus on the **total** Section B score (not a fixed weight):

$$\text{Section B} = 0.5 \times \text{Functional} + 0.25 \times \text{GitHub} + 0.25 \times \text{Video}.$$

Submit a **public GitHub** repository and put a clear link to your presentation video in the README.

Functional (50% of Section B) Mean NDCG@10 over 50 hidden queries (binary relevance). **Relative ranking** on the hidden set applies *inside* this component; you see only your own result after grading:

- 1st: 100% of this component + 5 bonus points on Section B.
- 2nd: 100% + 3 bonus points on Section B.
- 3rd: 100%.
- 4th+: $\min(1, \text{NDCG}_{\text{team}} / \text{NDCG}_{\text{3rd}}) \times 100\%$ of this component.

Missing or incompatible **artifacts**/ prevent running `run()` and yield 0 for this part.

GitHub repository (25% of Section B) Staff grade the repo on a 0–100 rubric, then scale to 25% of Section B:

- **50%** — seamless run: with dependencies already installed (no `pip install` at grading), `python scripts/eval_public.py` succeeds on a fresh clone **without** rebuilding the index.
- **20%** — modular code: small functions, logical split across files.
- **10%** — clear README (setup, artifacts, how to run eval).
- **10%** — clear in-code documentation.
- **10%** — evident pair collaboration: both members have meaningful commits in the history (not a single-author dump).

Video presentation (25% of Section B) **Requirements:** max **3:00**; at most **10 slides**; **both** team members speak; link in README. For **each** pipeline stage (**chunk**, **embed**, **index**, **retrieve**), explain your method. Show the **process** you followed and **empirical results** (plots/metrics) that justify your final design. Do **not** present by scrolling through code, a manuscript, or pasted code on slides.

Staff grade on a 0–100 rubric, then scale to 25% of Section B. **Penalties** (deducted from the video subscore only): 0.1 point per second over 3:00; 0.1 point per slide over 10.

- **30%** — clarity: we understand your method and pipeline choices.
- **20%** — empirical discussion: decisions backed by results you obtained during development (shown visually).
- **20%** — slides: concise and clear (within the 10-slide limit).
- **20%** — both members participate meaningfully.
- **10%** — time management (within 3:00, before penalties).

Creativity bonus Not part of the formula above. The **most creative** submission (staff judgment, informed mainly by the video) may receive an extra bonus on your **total** Section B score.